AI01 / EXPANDED FIELD CASE STUDY

AI-Assisted SOC Analyst

Carlos Tiburcio | November 2026 | Application baseline v0.7.0

I built AI01 to turn the telemetry in my homelab into investigations I could follow, save, and explain. It combines deterministic incident correlation with an analyst workspace and optional Azure AI analysis.

39 regression tests	3 / 3 live scenarios	3 telemetry families
Recorded final suite	DNS, Kerberos and remote SMB	Windows, Zeek and Suricata

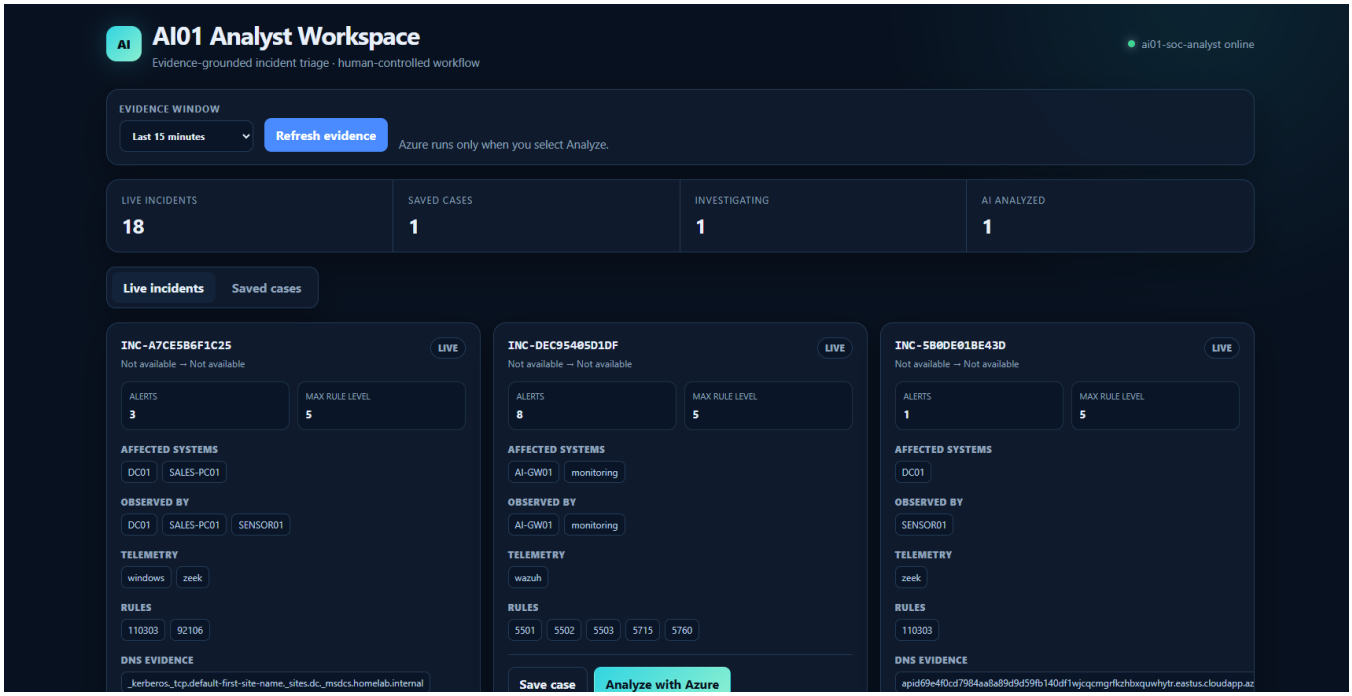


Figure 01. The working analyst dashboard: live incidents, affected systems, observers and explicit per-incident actions. Captured during Phase 5.

The result

AI01 retrieves security evidence through a read-only Wazuh account, groups related observations, and preserves case status and notes in SQLite. Azure returns structured investigation guidance only when requested. The analyst remains responsible for deciding what happened and what to do next.

This expanded case study follows the build from access and TLS troubleshooting through network enrichment, saved cases, service deployment and controlled evaluation. Screenshots are original project evidence; source-code explanations and reported checkpoints are identified separately.

AI01 / PURPOSE AND BUILD SEQUENCE

Why I built it, and how it grew

SOC01 already collected endpoint and network telemetry. The next problem was context: an alert list did not reliably show which records belonged together, which machine was affected, or what evidence an analyst should collect next. Repeating that work for every alert also made it difficult to maintain a consistent case history.

I wanted a practical investigation assistant that could explain its inputs. Correlation, evidence limits, asset identity and saved case state therefore live in ordinary application code. The model helps interpret a selected evidence set; it does not control collection or take action on endpoints.

Stage	What I built	Where to read
1. Foundation	AI-GW01, read-only index access, TLS handling and Python project structure	Pages 3-5
2. Azure analysis	Configured deployment and validated structured responses	Page 6
3. Correlation	Bounded incident candidates and traceable evidence relationships	Page 7
4. Network context	Zeek indexing repair, DNS rules and asset-aware enrichment	Pages 8-10
5. Analyst workflow	Dashboard, SQLite cases, status, notes and saved analysis	Pages 11-12
6. Service operation	Dedicated identity, systemd hardening, health checks and recovery	Pages 13-14
7. Live evaluation	Known DNS, Kerberos and SMB activity checked against explicit criteria	Pages 15-19
8. Documentation	Selected evidence, engineering lessons and portfolio publication	Page 20

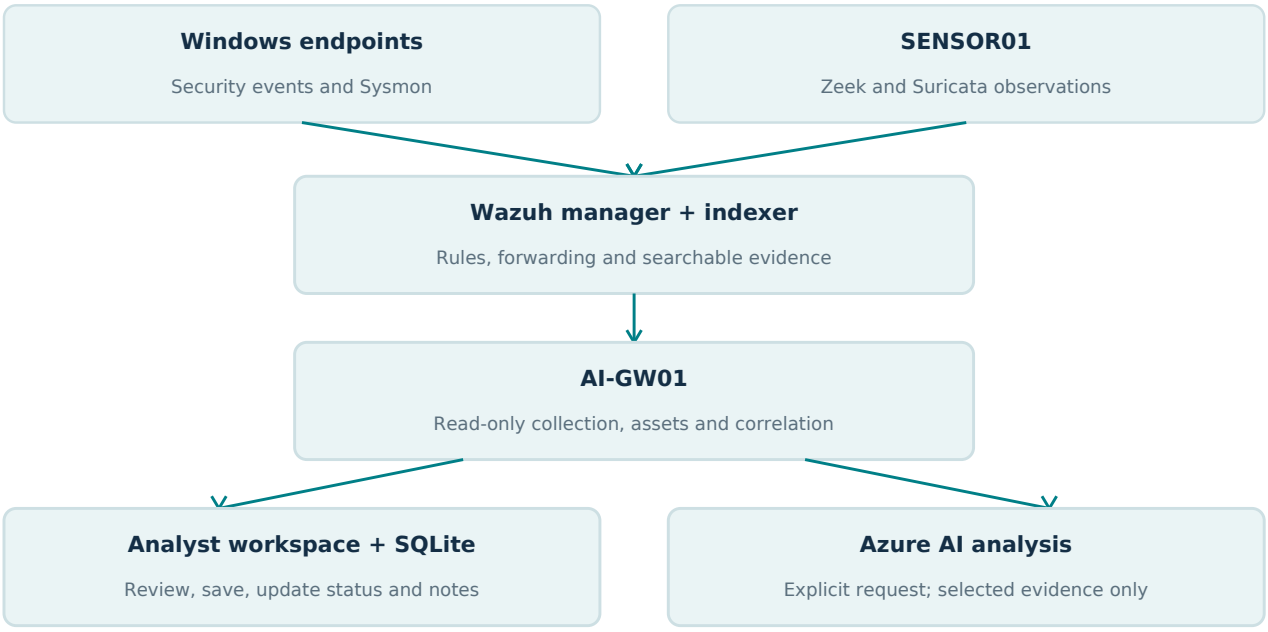
What this project demonstrates

The work connects security monitoring, Python API development, Linux administration, cloud AI integration, detection tuning and investigation writing. The strongest part is the evidence path: I can show how a raw event became an incident and why a proposed conclusion still needs verification.

Scope: This is a completed homelab application baseline. The three live exercises are controlled tests, not confirmed compromises or a general accuracy benchmark.

AI01 / SYSTEM DESIGN

Architecture and lab inventory



The Wazuh manager evaluates incoming telemetry and the indexer makes alert documents searchable. AI-GW01 queries those documents, normalizes their fields, resolves known assets, and forms incident candidates. The analyst can save a candidate and request Azure analysis of its bounded evidence. [S1, S2]

System	Address used in AI01	Role
AI-GW01	10.10.30.120	Ubuntu application gateway
WAZUH01 / monitoring	10.10.30.100	Wazuh manager and indexer
SENSOR01	10.10.30.110	Zeek and Suricata observer
DC01	10.10.20.100	Active Directory and approved DNS
FILE01	10.10.20.120	Windows file server
IT-PC01	10.20.10.121	Administrative test workstation
SALES-PC01	10.20.20.168	DNS test workstation

Addresses above come from the AI01 asset inventory and live evidence. They describe this project baseline; workstation DHCP addresses may change. The underlying environment is the existing Proxmox lab with OPNsense routing.

Trust boundaries: Wazuh access is read-only. The web application binds to loopback and is reached through SSH forwarding. Azure receives selected evidence on an explicit analysis request. Local case updates do not modify the SIEM or endpoints.

AI01 / FOUNDATION / IDENTITY AND INGESTION

Give the collector only the access it needs

I separated the evidence reader from administrative accounts. The application needs to search alert documents, so I mapped **ai01_reader** to an index-reader role scoped to **wazuh-alerts-***. This makes the permission boundary visible instead of relying on a promise in the AI prompt.

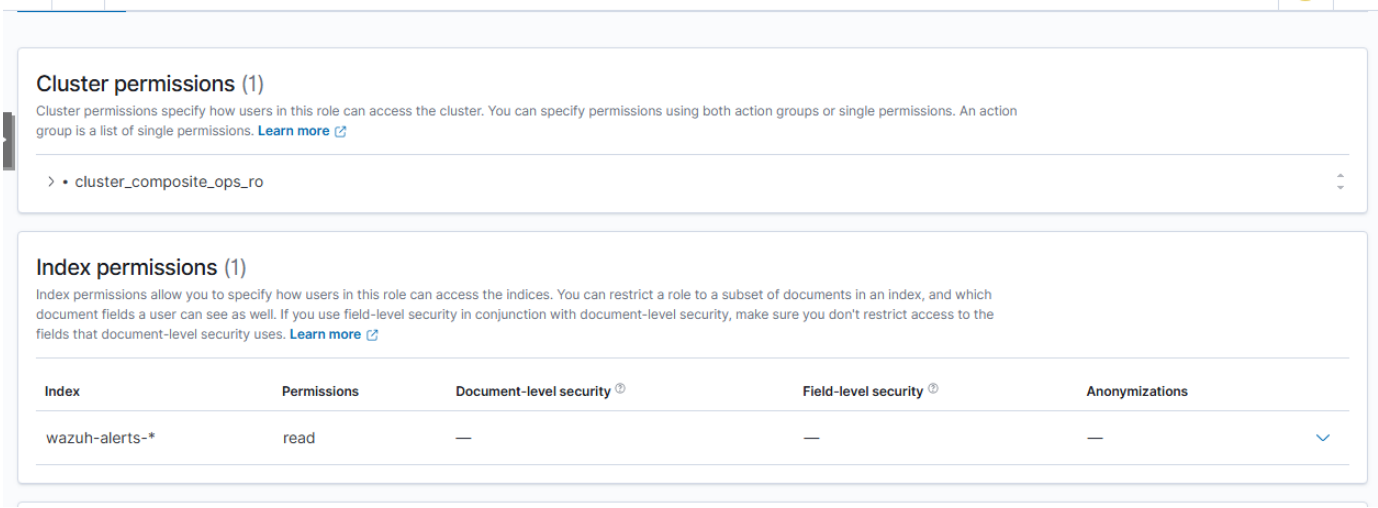


Figure 02. The Wazuh index role grants read access to wazuh-alerts-* with read-only composite cluster operations.

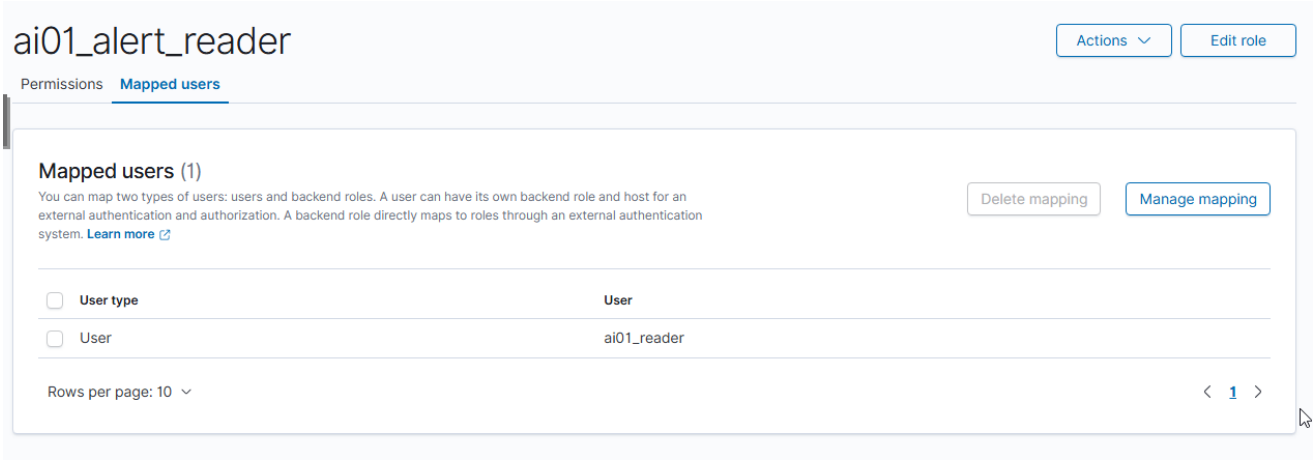


Figure 03. The ai01_alert_reader role is mapped to the dedicated ai01_reader user.

A working API token was only the first connection check

The final evidence collector queries the **Wazuh indexer** search endpoint. Manager API authentication and indexed-alert retrieval are different checks. I verified access to the searchable documents that the application actually consumes, then normalized them before returning them through FastAPI.

Collection limits are enforced in code: 1-100 alert documents and, for correlation, a 1-1,440 minute lookback. Queries reserve space for Zeek and Suricata so a busy stream of Windows records does not hide network evidence. The returned merged set stays within the requested limit. [S2]

AI01 / FOUNDATION / TROUBLESHOOTING

Resolve TLS and verify the Python baseline

The indexer certificate named 127.0.0.1 rather than its production address. That explained why a normal hostname check could not validate the connection from AI-GW01. I inspected the certificate instead of treating every connection failure as a firewall problem.

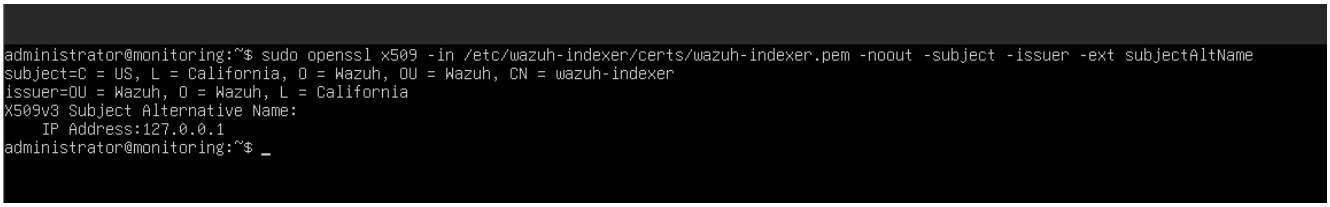


Figure 04. Certificate inspection shows the Subject Alternative Name is 127.0.0.1.

The implemented interim control requires HTTPS and pins the exact server public key with SPKI through pycurl. Conventional CA-chain and hostname verification are disabled in this collector; the pin is the server identity check. Reissuing a correctly named certificate and enabling normal verification remains the cleaner long-term configuration. [S2]

Keep the project environment reproducible



Figure 05. Both Python and pytest resolve inside the project virtual environment.

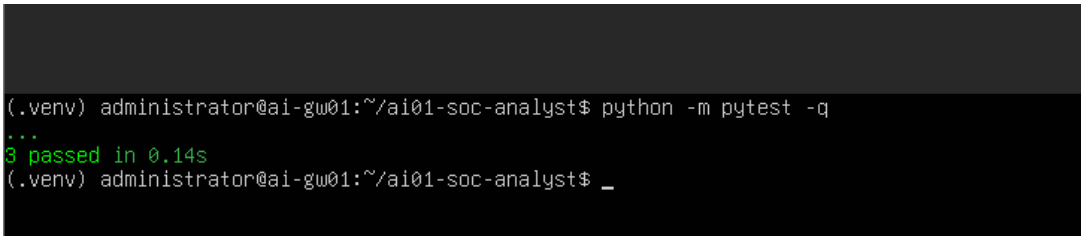


Figure 06. The initial application test checkpoint passed five tests. This is an early milestone, not the final suite.

The project used a virtual environment, tracked dependencies, a local environment file excluded from Git, and feature branches for changes. I used **python -m pytest** so test execution followed the selected interpreter. Later checkpoints grew to 25, 29, 33 and finally 39 tests as functionality expanded.

> **Engineering lesson:** Connection reachability, credentials, certificate identity and application parsing are separate layers. Testing them independently made failures easier to locate.

AI01 / AZURE INTEGRATION

Connect Azure and ask for structured analysis

I configured an Azure-hosted deployment named **ai01-gpt-5-mini** and loaded its endpoint, deployment name and API key from local configuration. The application uses a validated response schema rather than accepting an unrestricted block of prose. [S3]

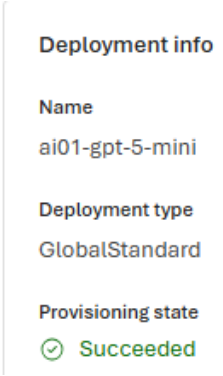


Figure 07. Azure deployment detail: the named deployment is provisioned successfully. Cropped to the deployment fields.

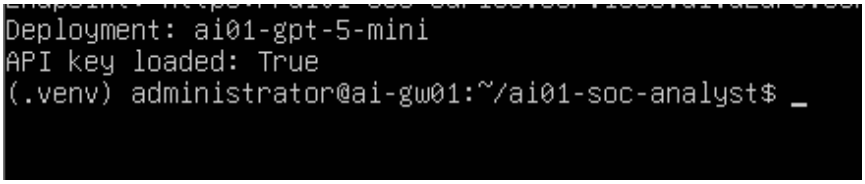


Figure 08. Local configuration check confirms the deployment name and that a key was loaded; the key value is not displayed.

What goes into a request

The analyzer sends document IDs, timestamps, rule information, observer identity, users, IPs and normalized network context. It rejects an empty batch and accepts no more than 25 alerts per call. Azure configuration is checked before the request, and an unusable or unparsed response is reported as an analysis error.

Output	Purpose
Facts and hypotheses	Separate observations from possible explanations
ATT&CK; suggestions	Provide technique IDs, confidence and rationale
Investigation steps	Name an ordered action and the data source to inspect
Missing evidence and references	Keep uncertainty and source documents visible

The request uses store=False. That request setting is not a claim about every provider retention policy. The important application boundary is that opening the incident list or checking health does not automatically invoke Azure.

AI01 / DETERMINISTIC CORRELATION

Build incidents before asking the model

The first correlation work proved that the application could produce an incident and analyze it end to end. It also exposed why simply grouping shared IPs and agent names was too broad: background network anomalies and SMB events could end up in the same early candidate.



Figure 09. Early development response: HTTP 200, nine alerts and recorded correlation reasons. The displayed systems field predates the final affected-system / observer separation.

How the final correlation became more selective

Control	Reason
Bounded time window	Relationships must fit the configured incident window.
Flow and service context	Avoid joining unrelated services solely because they share an IP.
High-signal identity	Generic identities such as NT AUTHORITY\SYSTEM remain visible but do not act as broad user-linking keys.
Address filtering	Loopback, link-local, multicast and other unsuitable addresses are excluded as relationship keys.
Stable evidence-based IDs	Incident identifiers derive from the selected evidence IDs, making an exact evidence set traceable.
Priority beyond raw volume	Multi-system and multi-source evidence is considered when ranking candidates.

The later SMB evaluation added one narrow relationship: rule 110207 and rule 92106 on the same agent, within 30 seconds, with a meaningful user in the privileged-logon record. That correction is documented with the failed and passing tests on pages 17-18. [S4]

Important distinction: A repeatable correlation algorithm can still produce an incomplete or misleading candidate. Live scenarios test whether the selected relationships hold up against known activity.

AI01 / NETWORK TELEMETRY / CAPTURE AND RETRIEVAL

Follow a DNS event through the pipeline

I generated a recognizable DNS query from SALES-PC01 and followed it through Zeek, Wazuh and the indexer. This was more useful than checking whether each service simply reported itself as running.

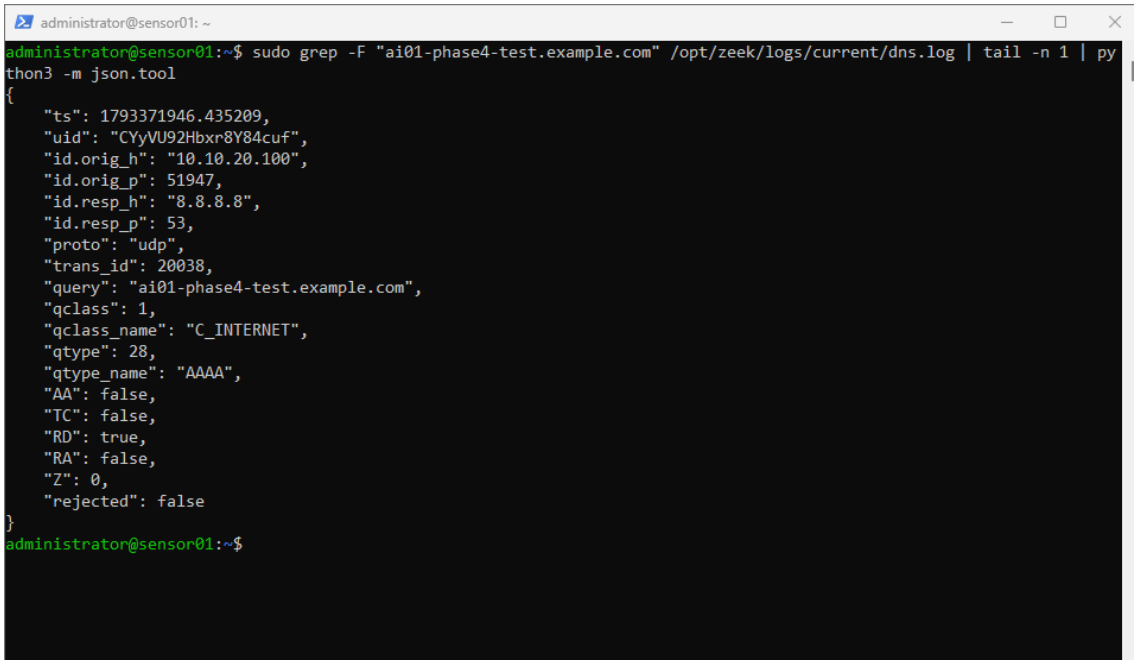


Figure 10. The controlled DNS marker appears in Zeek dns.log with the originating workstation, external resolver, ports and query.

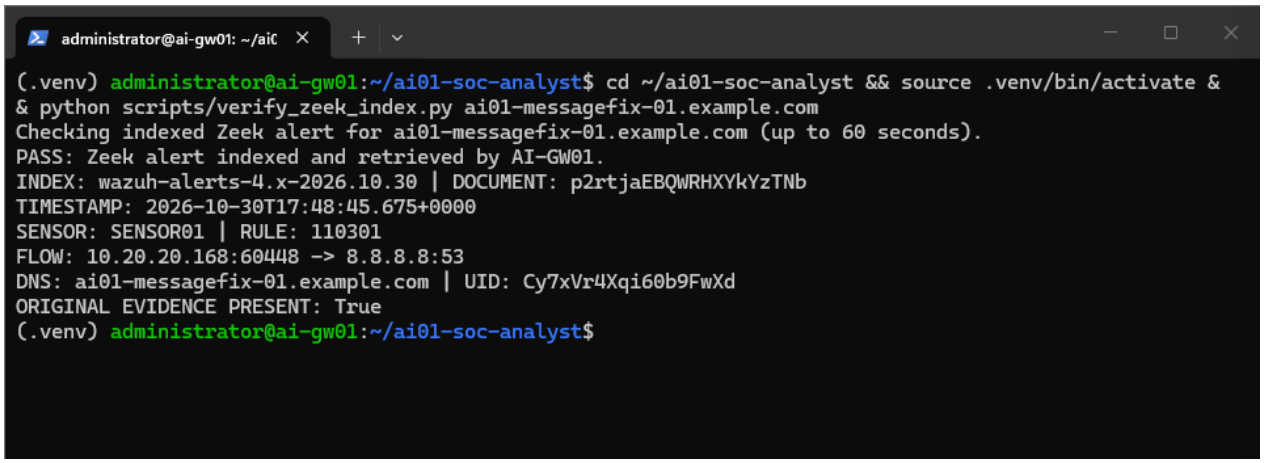


Figure 11. After the indexing repair, AI-GW01 retrieves the exact marker and reports the rule, sensor, flow, UID and original evidence.

Earlier capture troubleshooting included enabling Zeek ignore_checksums, consistent with checksum-offload artifacts in mirrored VM traffic. Forwarding was observed working after the localfile format changed to syslog. Those observations identify successful checkpoints; they do not prove every internal cause of the earlier failures. [S5]

The successful index verification is an ingestion result. It proves the event can be queried and its fields survived; AI interpretation was a separate validation step.

AI01 / NETWORK TELEMETRY / ROOT CAUSE

Repair the Zeek indexing mismatch

The DNS event existed in Wazuh local logs but did not initially appear in the searchable alert index. The cause was a schema conflict: Zeek uses dotted connection keys, Wazuh decoded them into an object under **data.id**, and the existing index mapping expected **data.id** to be a keyword. [S5]

Why the first processor approach was ineffective

The Wazuh Filebeat module was still carrying the serialized alert in its **message** field. The indexer pipeline decoded it later. A processor aimed at root-level data.id or location therefore ran before those fields existed at that level.

Decoded Zeek field	Indexed field
data.id.orig_h	data.src_ip
data.id.orig_p	data.src_port
data.id.resp_h	data.dest_ip
data.id.resp_p	data.dest_port

The scoped repair

I used a Filebeat JavaScript processor to parse the serialized message, require a known Zeek log location and the zeek rule group, validate all four connection fields, and move them into the compatible names above. It removes the conflicting object only after those checks pass.

The processor preserves the original full_log, query, UID, rule metadata, timestamp and alert ID. Unexpected input is left intact and tagged as a normalization error; it is not silently dropped. Other alert families pass through unchanged.

Validation	What was checked
Before deployment	Filebeat configuration and built-in processor tests passed; seven diagnostic Zeek records were normalized while three sudo records stayed unchanged.
Fresh live event	The marker query was found in the index by verify_zeek_index.py, including normalized flow fields and original evidence (Figure 11).
Historical events	Previously rejected documents were not automatically replayed. A fresh query was necessary to validate the change.

What I learned: A rule firing in alerts.json is not proof that the document reached the index. Capture, rule evaluation, forwarding, indexing and application retrieval each need their own evidence.

AI01 / DETECTION AND ENRICHMENT

Tune DNS signals and preserve asset identity

A long DNS name can be legitimate, especially for cloud services. I reduced the standalone long-query signal to level 5 and used the combination of query length and resolver bypass for higher-priority escalation. DC01 remains the approved resolver in these lab rules. [S6]

Rule	Level	Meaning in this lab
110300	0	Base classification for DNS records
110301	7	Direct DNS query where neither endpoint is DC01
110306	9	Resolver bypass plus a query at least 60 characters long
110302	5	DNS request marked rejected
110303	5	Long query requiring context; length alone is insufficient
110304 / 110305	0 / 7	DNS connection classification / direct attempt with zero observed response packets

The sensor is not the affected workstation

A Wazuh agent name of SENSOR01 identifies the observer for Zeek and Suricata evidence. I used the versioned asset inventory to resolve 10.20.20.168 to SALES-PC01, then kept affected systems and observing systems in separate fields.

Field	DNS example	Interpretation
Source asset	SALES-PC01	Workstation originating the query
Destination	8.8.8.8:53	External DNS resolver
Observed by	SENSOR01	Sensor that recorded the traffic
Rule metadata	T1071.004	Investigation category, not a finding of command and control

Evidence limit: No response seen by a passive sensor does not prove a firewall block or that the request failed everywhere. A client NXDOMAIN result is a DNS response. I kept those distinctions explicit in the analysis prompt and documentation.

AI01 / ANALYST WORKSPACE / SAVED CASES

Make the investigation persistent

The dashboard turned the API into a usable workflow: choose an evidence window, inspect an incident, save it, and record status and notes. A saved snapshot remains available after the live Wazuh query window moves on. [S7]

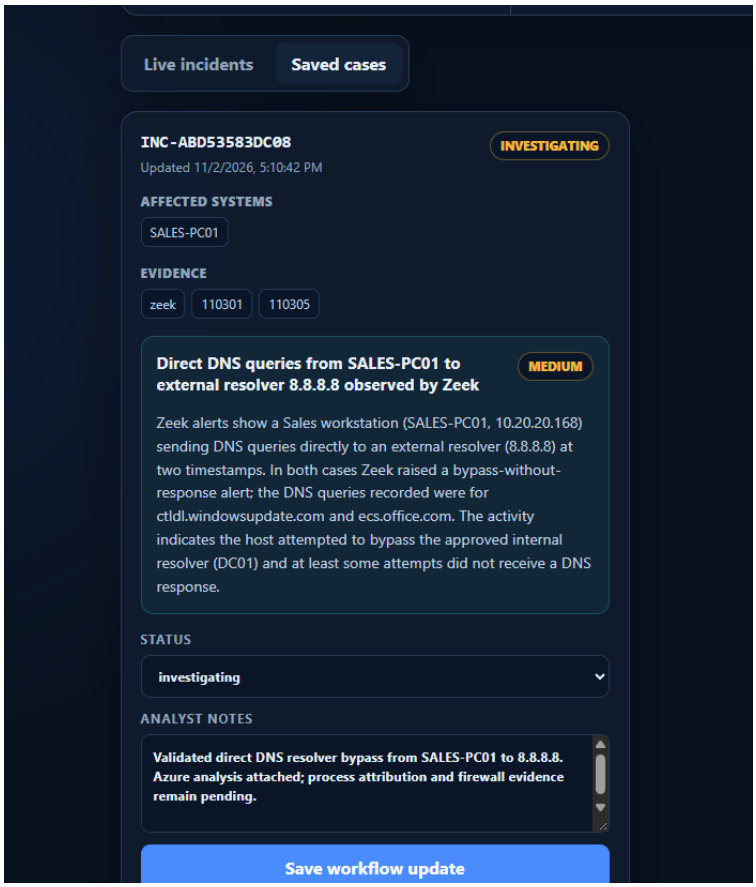


Figure 12. Saved case in investigating state with analyst notes, affected system and an attached analysis summary. Cropped to the case panel.

Analyst action	Application behavior
Save an incident	Persist the selected incident snapshot in SQLite
Update status or notes	Keep analyst-owned case state across later reads
Analyze selected incident	Attach structured Azure output and its timestamp
Reopen a saved case	Retrieve persisted evidence, workflow state and analysis

Analysis does not automatically close a case. The Phase 5 validation also checked that attaching a new Azure result preserved the existing status and analyst notes. That matters because model output should add context without erasing investigation work.

AI01 / ANALYST WORKSPACE / STRUCTURED OUTPUT

Make uncertainty visible in the interface

I expanded the saved-case view beyond a title and summary. It exposes the model's facts, hypotheses, ATT&CK; suggestions, investigation steps and missing evidence so an analyst can review the reasoning against the source documents.

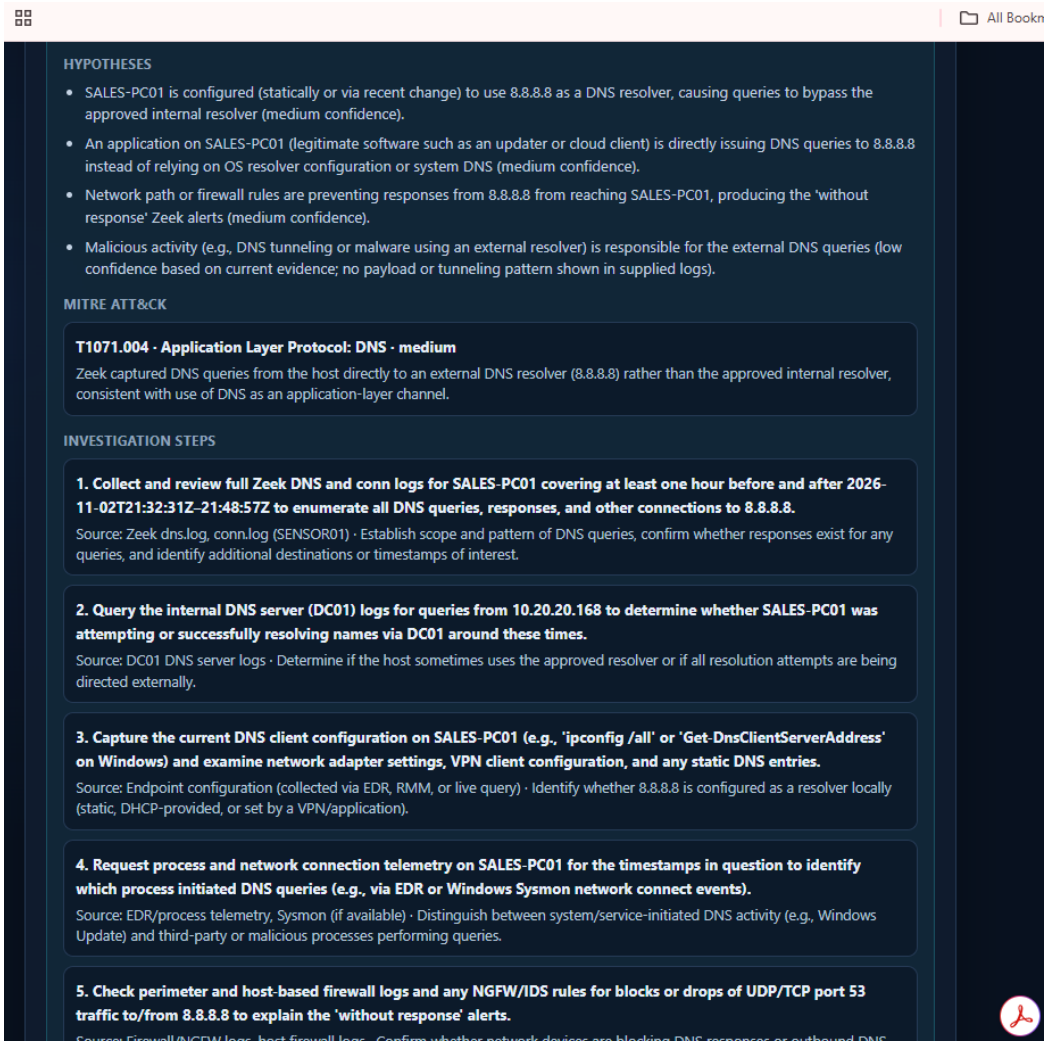


Figure 13. Structured case analysis shows hypotheses, ATT&CK; context and specific investigation steps with named data sources.

The DNS case considers configuration and legitimate application behavior alongside a tunneling hypothesis. The investigation steps point to workstation resolver settings, network flows, Windows events and the relevant Zeek records. These are leads to verify, not completed investigative actions.

Review finding: Schema validation guarantees the expected output shape, not factual correctness. In one saved DNS summary, missing sensor response evidence was phrased more strongly as the client not receiving a response. An analyst should retain the narrower claim supported by the sensor.

AI01 / DEPLOYMENT AND HARDENING

Run AI01 as a service

Once the workflow worked interactively, I moved it into a dedicated service installation. Code, secrets and writable case data have separate locations and owners. The service runs as the non-login **ai01** account rather than my administrative shell. [S8]

Purpose	Path	Owner / mode
Application	/opt/ai01	root:ai01; dirs 0750, files 0640
Runtime configuration	/etc/ai01/ai01.env	root:ai01; 0640
SQLite case store	/var/lib/ai01/ai01.db	ai01:ai01; 0600
Service and timer	/etc/systemd/system/	root:root; unit files 0644

The deployed service contract

```
User=ai01
Group=ai01
WorkingDirectory=/opt/ai01
EnvironmentFile=/etc/ai01/ai01.env
Restart=on-failure
RestartSec=5s
UMask=0077
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/ai01
```

Excerpt from the committed service unit, not a terminal screenshot. The full unit also drops Linux capabilities and applies private temporary storage, device restrictions and kernel protections. Uvicorn uses one worker and listens only on 127.0.0.1:8000. [S8]

Access and health monitoring

```
ssh -L 8000:127.0.0.1:8000 administrator@10.10.30.120
```

The analyst opens the local forwarded address while the SSH session stays connected. A separate timer runs the localhost health check every two minutes and writes results to journald. The loopback-only design avoids treating Uvicorn as a public, authenticated web service.

AI01 / OPERATIONAL VALIDATION

Test recovery, not just startup

A service can start successfully and still lose case data or fail to recover after a crash. Phase 6 tested those behaviors explicitly. The table records the outcomes documented in the project README and deployment notes; it is not a new run against the live lab. [S1, S8]

Check	Recorded outcome
Migration integrity	Source and production database SHA-256 values matched.
Permissions	Secrets were root:ai01 0640; the case database was ai01:ai01 0600.
Forced process failure	systemd restarted the process after five seconds.
State after process failure	The saved case and attached Azure analysis remained available.
Full AI-GW01 reboot	Application and health timer started automatically.
State after reboot	Case and analysis persisted; health and case APIs returned HTTP 200.
Health-check timing	An initial connection-refused result raced startup; later checks succeeded.

Regression checkpoints

Milestone	Recorded passing tests	Coverage added
Foundation	5	Configuration, collector and basic API
Phase 4	25	Network normalization, assets and bounded correlation
Phase 5	29	Saved cases, workflow state and dashboard
Phase 6	33	Deployment and service health behavior
Phase 7	39	Scenario evaluation and narrow SMB correlation

The final 39-test result is the recorded project checkpoint. Automated tests checked application behavior; the controlled scenarios on the next pages checked real telemetry. Neither substitutes for the other.

Boundary: A local health endpoint confirms that the application answers. It does not by itself prove fresh Wazuh ingestion, usable Azure credentials or valid model conclusions. Those dependencies require their own checks.

AI01 / CONTROLLED EVALUATION / 7 OF 7

Live test 1: direct DNS resolver bypass

On SALES-PC01, I queried 8.8.8.8 directly using a recognizable long DNS marker. This exercised the resolver policy, Zeek capture, Wazuh rule 110306, normalized fields, asset mapping and the incident selector together.

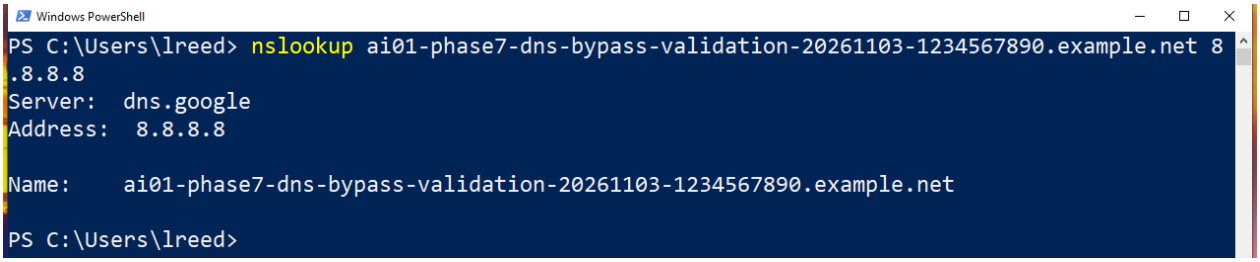


Figure 14. Controlled query from SALES-PC01 to 8.8.8.8. The client reports a nonexistent domain; that response does not establish command and control.

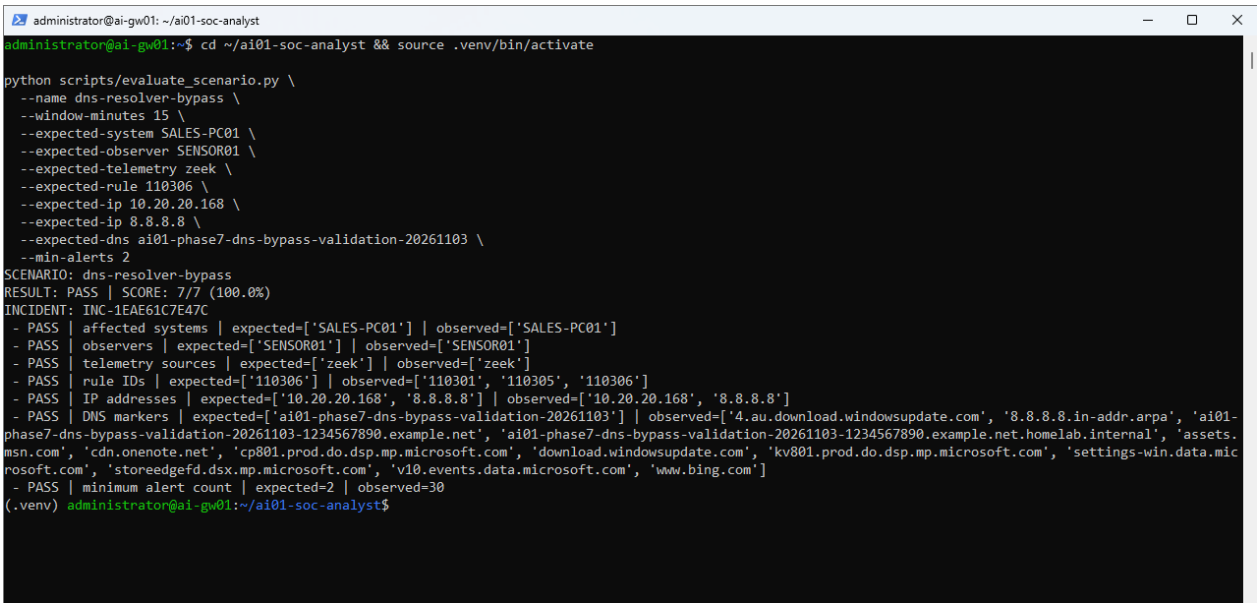


Figure 15. The evaluator selects the matching DNS incident and passes all seven checks, including the DNS marker, endpoint IPs, Zeek source and SENSOR01 observer.

How the score was calculated

The evaluator checks expected systems, observers, telemetry families, rule IDs, IPs, DNS markers and minimum alert count. Expected values are subset checks: additional related evidence can remain present. It selects the candidate satisfying the most explicit criteria, rather than assuming the first incident is the intended test. [S9]

The pass confirms that the expected DNS evidence was present in the selected incident. It does not prove every alert in that incident was causally related or that the activity was malicious.

> **Observed outcome:** SALES-PC01 was the affected asset and SENSOR01 the observer. The model treated DNS application-layer behavior as an investigation lead and did not claim confirmed command and control.

AI01 / CONTROLLED EVALUATION / 6 OF 6

Live test 2: Kerberos failure burst

I generated five incorrect-password attempts for **det01.user01** from IT-PC01. DC01 produced the single-failure and threshold alerts created in the earlier DET01 project. AI01 needed to preserve both rule IDs and the user identity in one candidate.

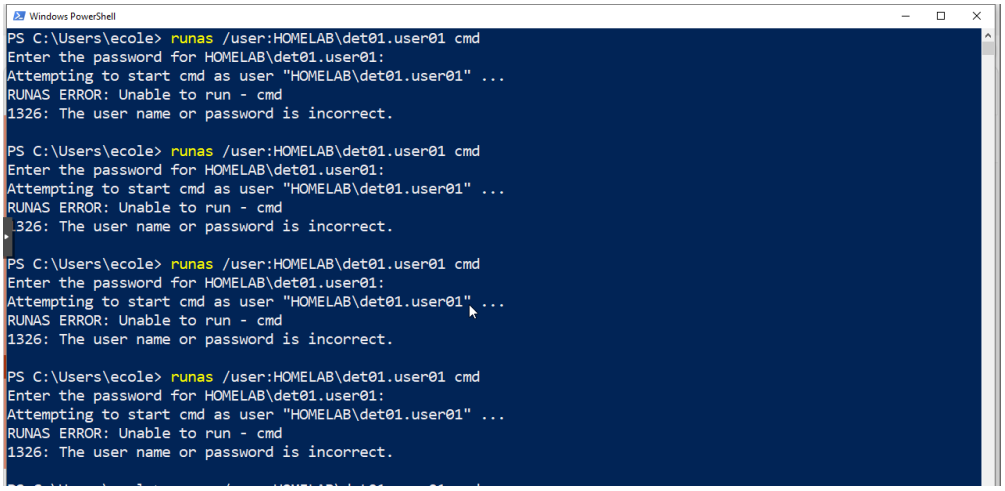


Figure 16. The controlled Windows authentication attempts return incorrect-password messages.

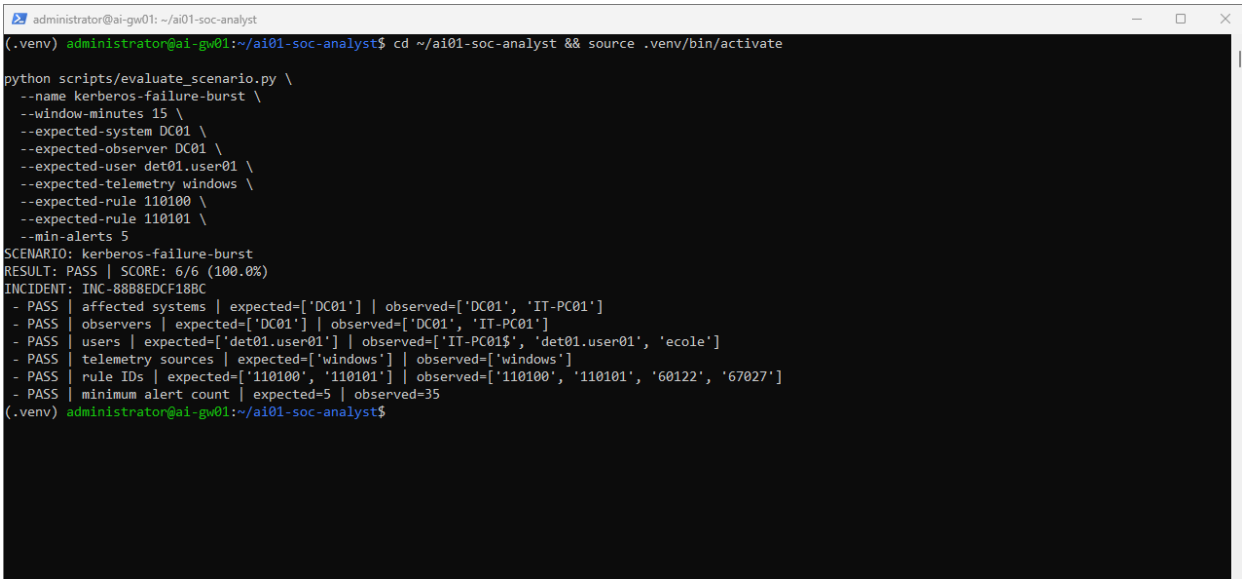


Figure 17. The Kerberos evaluator passes 6/6, retaining DC01, det01.user01 and rules 110100 / 110101.

The selected incident contained more than the five generated failures because other nearby Windows evidence was present. The test required at least five alerts and checked that the expected identity, observer, source family and rule IDs were included. It did not require an otherwise quiet environment. [S9]

> **Observed outcome:** AI01 found the expected authentication pattern inside live background telemetry. This validates the chosen lab scenario, not general password-spray detection coverage.

AI01 / CONTROLLED EVALUATION / FINDING AND CORRECTION

Live test 3: investigate the failed SMB score

From IT-PC01, I used known Administrator credentials to access FILE01's administrative share. The share access succeeded, but the incident evaluator initially failed. That was a useful distinction: the test activity worked while the assembled incident was missing expected identity evidence.

```
PS C:\Users\ecole> Get-ChildItem \\FILE01\C$ | Select-Object -First 5


    Directory: \\FILE01\C$


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          5/8/2021   4:20 AM                PerfLogs
d-r---          9/13/2026   1:31 PM                Program Files
d-----         10/13/2026   8:14 PM                Program Files (x86)
d-----         10/13/2026   8:23 PM                SOC01
d-r---          9/13/2026   1:43 PM                Users


PS C:\Users\ecole> net use \\FILE01\C$ /delete
\\FILE01\C$ was deleted successfully.

PS C:\Users\ecole>
```

Figure 18. Authorized C$ access and directory listing succeed on FILE01, followed by removal of the mapping.

```
administrator@ai-gw01: ~/ai01-soc-analyst
(.venv) administrator@ai-gw01:~/ai01-soc-analyst$ python scripts/evaluate_scenario.py \
  --name authorized-remote-admin-smb \
  --window-minutes 15 \
  --expected-system FILE01 \
  --expected-observer FILE01 \
  --expected-user Administrator \
  --expected-telemetry windows \
  --expected-rule 110207 \
  --expected-rule 92106 \
  --min-alerts 2
SCENARIO: authorized-remote-admin-smb
RESULT: FAIL | SCORE: 4/6 (66.7%)
INCIDENT: INC-8ADBDC31BBA8
 - PASS | affected systems | expected=['FILE01'] | observed=['FILE01']
 - PASS | observers | expected=['FILE01'] | observed=['FILE01']
 - FAIL | users | expected=['Administrator'] | observed=['NT AUTHORITY\\\\SYSTEM']
 - PASS | telemetry sources | expected=['windows'] | observed=['windows']
 - FAIL | rule IDs | expected=['110207', '92106'] | observed=['92110']
 - PASS | minimum alert count | expected=2 | observed=4
(.venv) administrator@ai-gw01:~/ai01-soc-analyst$
```

Figure 19. A later pre-pass evaluation scores 4/6: Administrator and the privileged-logon rule are missing from the selected incident.

The Sysmon SMB-port alert named **NT AUTHORITY\SYSTEM**, while a separate privileged-logon record carried the actual Administrator identity. I inspected the indexed records and added a deliberately narrow relationship: exact rules **110207 + 92106**, same agent, no more than **30 seconds** apart, with a high-signal user on the privileged record. [S4, S9]

The failed check stayed in the project record. I did not weaken the expectations to turn an incomplete incident into a passing result.

AI01 / CONTROLLED EVALUATION / 6 OF 6

Repeat SMB with fresh authentication

Deleting a drive mapping did not always produce a fresh privileged-logon event because Windows could reuse authentication state. I cleared mappings and Kerberos tickets, connected using FILE01's IP address, then allowed the SIEM pipeline time to index the new evidence.

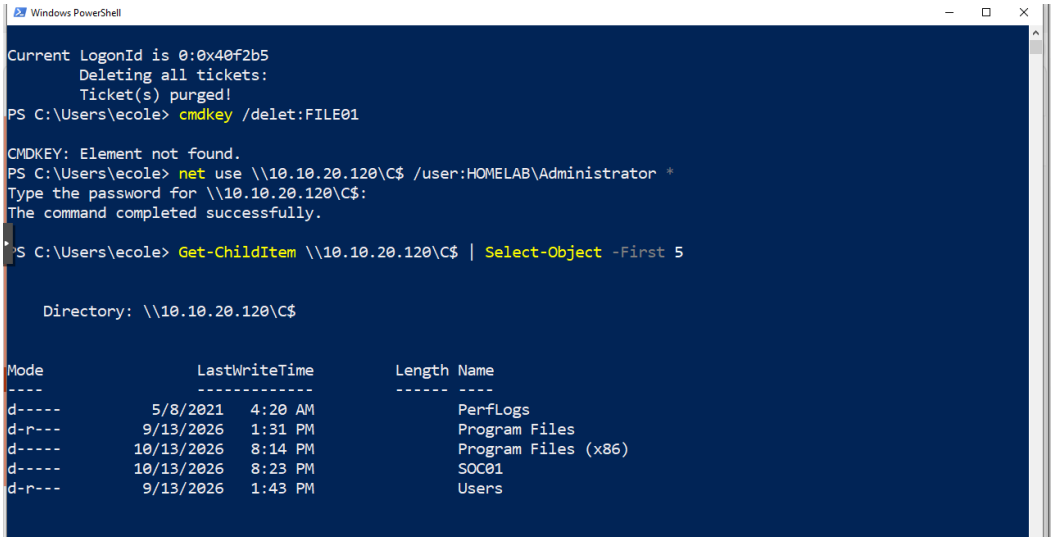


Figure 20. The repeat exercise clears cached Kerberos tickets and accesses FILE01 by IP before removing the mapping.

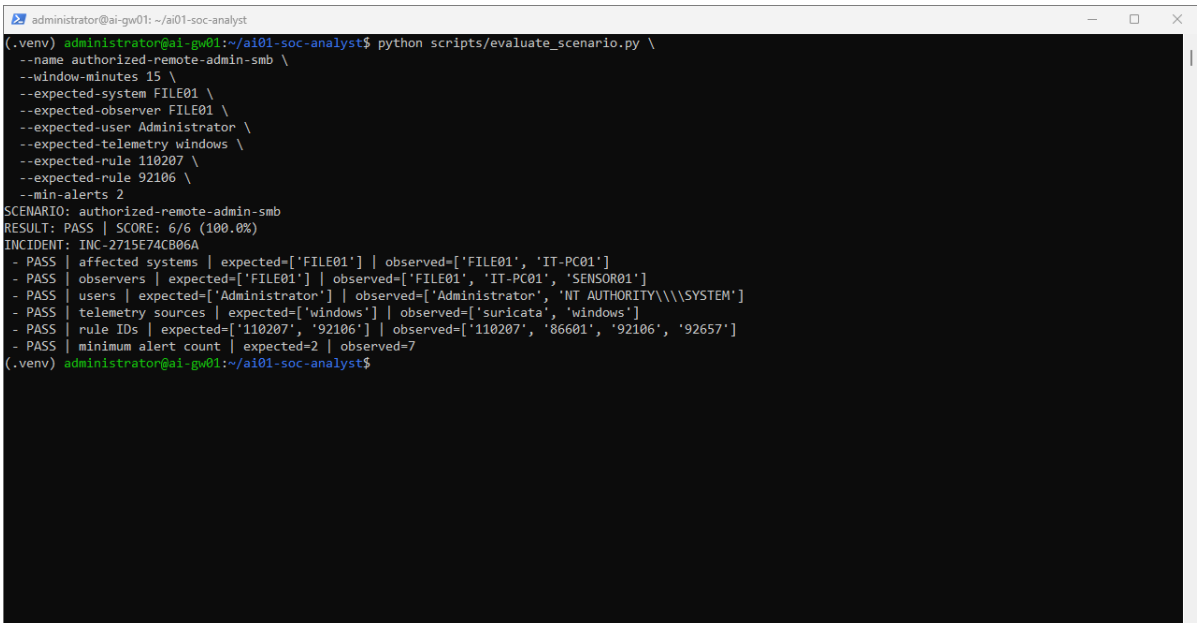


Figure 21. Final SMB evaluation: 6/6. FILE01 and IT-PC01 appear as affected systems, while SENSOR01 is retained as an observer.

The selected incident contained seven alerts, the Administrator identity, rules 110207 and 92106, and both Windows and Suricata evidence. The required telemetry check was Windows; Suricata appeared as supporting evidence in the final observed result. [S9]

I allowed two to three minutes for Wazuh processing in the final procedure. That delay is a testing constraint: an evaluation run before indexing finishes can select a less complete candidate.

AI01 / SMB ANALYSIS / ANALYST REVIEW

Read the AI result against the evidence

I analyzed the passing SMB incident explicitly. The response rated it high for investigation and described NTLM/SMB traffic alongside a privileged Administrator logon. It considered authorized administration and unauthorized lateral movement as competing explanations.

```
HTTP: 200
INCIDENT: INC-2715E74CB06A | ALERTS: 7
TELEMETRY: ['suricata', 'windows']
AFFECTED SYSTEMS: ['FILE01', 'IT-PC01']
OBSERVED BY: ['FILE01', 'IT-PC01', 'SENSOR01']
IPS: ['10.10.20.120', '10.20.10.121']
DNS: []
RULES: ['110207', '86601', '92106', '92657']
AI TITLE: Investigation: NTLM/SMB authentication and privileged Administrator logon on FILE01
SEVERITY: high
```

Figure 22. Selected-incident analysis returns HTTP 200 and identifies affected systems, observers, rules and the investigation title. Excerpt from the original terminal output.

Supported observation	Conclusion still requiring evidence
NTLM/SMB exchanges and Windows authentication alerts	Which NTLM version was actually negotiated
Privileged Administrator logon on FILE01	Whether that access was approved outside this controlled test
System-process SMB network activity	Which initiating process and user action caused each connection
Detection text suggesting possible pass-the-hash	Credential theft, hash replay or a confirmed pass-the-hash technique

The output suggested checking 4624, 4648 and 4672 events, share-access records, endpoint process telemetry and packet evidence. Those are investigation recommendations, not evidence that those records were already collected.

Review finding: The response preserved uncertainty about pass-the-hash, but one ATT&CK; technique name was imprecise. Structured output and plausible technique IDs still require analyst review. The underlying test used authorized credentials; it did not demonstrate credential theft.

AI01 / OUTCOME AND EVIDENCE REFERENCES

What I learned and what remains bounded

The most useful work happened at the boundaries: a valid local alert that could not be indexed, an observing sensor mistaken for an affected asset, an SMB connection without its privileged identity, and a health check that ran before the process was ready. Fixing those gaps made the AI output more useful because the evidence underneath it became more reliable.

AI01 now gives me a repeatable path from alert documents to a saved investigation. It combines the monitoring built in SOC01, the authentication rules from DET01, privileged-logon context from ID01, and a Python/Azure application that I can operate and explain.

Limits and follow-up work

The evaluation covers three known lab scenarios. It does not establish precision, recall, a false-positive rate, general model accuracy or readiness for an enterprise deployment. SPKI pinning remains an interim certificate control. A local asset inventory needs maintenance as addresses change. The final Proxmox snapshot and a GitHub release/tag are separate operational actions; this PDF does not certify their completion.

Source and evidence guide

Original screenshots were selected from the AI01 build and evaluation captures dated October 28-November 4, 2026. Images are cropped only for focus and readability. Screenshot counts, timestamps and intermediate failures are preserved as captured. Deployment/recovery outcomes without a selected screenshot are attributed to the recorded repository checkpoints.

[S1] Project overview and validation checkpoints - README.md

[S2] Read-only collector and TLS behavior - app/collectors/wazuh.py

[S3] Azure prompt, request bounds and structured parsing - app/services/azure_analysis.py

[S4] Incident correlation logic - app/services/correlation.py

[S5] Zeek/Filebeat diagnosis and field repair - deploy/filebeat/README.md

[S6] Lab DNS detection rules - deploy/wazuh/ai01_zeek_rules.xml

[S7] Case persistence implementation - app/services/storage.py

[S8] Deployment and recovery procedures - deploy/systemd/README.md

[S9] Controlled scenario methodology and results - docs/phase-7-evaluation.md

Portfolio case study | Application repository